PTI — AGENTS LAB

A Network of Trusted AI Agents

From a single RAG chatbot to a network of agents that verify each other's identity and permissions before talking — using DIDs and Verifiable Credentials.

WHY

Why should agents trust each other?

AI agents are starting to call other agents' tools automatically, without a human approving each call. Before an orchestrator hands off a task, it needs a way to answer two questions:

"Who are you, really?"

→ solved with a **Decentralized Identifier (DID)**: a cryptographic identity an agent generates for itself.

"What are you allowed to do?"

→ solved with a **Verifiable Credential (VC)**: a signed statement, from a trust anchor, listing what that identity may do.

This lab builds exactly this, from scratch and locally: you'll first meet RAG (1) and MCP (2) — two ways an LLM chatbot reaches beyond its training data — then turn those chatbots into agents with real cryptographic identities that only act on requests they can verify (3).

ROADMAP

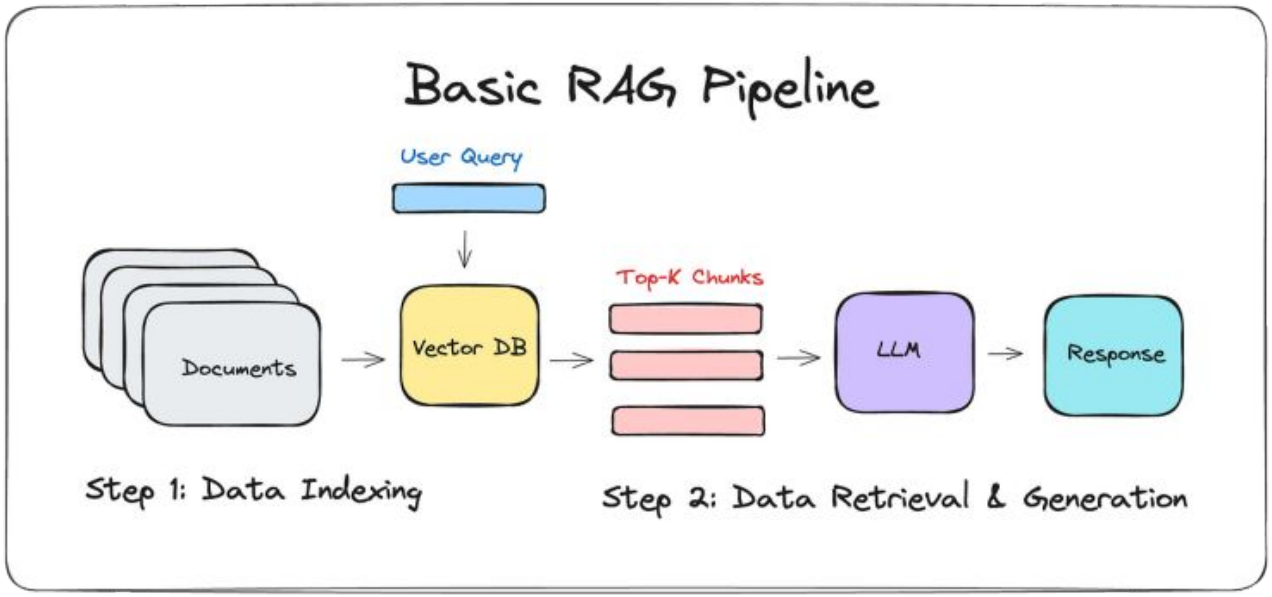
Required core, plus optional extensions

0	**Environment setup** Install Ollama, pull a chat model and an embedding model	REQUIRED
1	**RAG chatbot** Answer questions from a car-rental CSV via a vector index	REQUIRED
2	**MCP chatbot** Let the LLM call a live external tool instead of a static index	REQUIRED
3	**Trusted agent network** Wrap both capabilities as agents with DIDs + credentials; an orchestrator verifies before routing	REQUIRED
4	**Proof-of-possession** Prove an agent really controls its private key, not just its DID	OPTIONAL
5	**Self-proposed extension** Revocation, expiry, a 4th agent, mutual auth, ...	OPTIONAL

Retrieval-Augmented Generation (RAG)

An LLM only knows what was in its training data — it has never seen your rentals.csv. RAG bridges that gap:

- An embedding model turns text chunks into vectors that capture meaning.
  - nomic-embed-text builds the vector index once — llama3.2 can't embed, only chat.
  - Your **dataset (rentals.csv)** is indexed this way into a vector store, saved to disk.
  - At query time: your question is embedded too, the most similar chunks are retrieved, and only those are handed to the chat LLM as context.
  - **Answers stay grounded in real data instead of the model guessing**.



The RAG pipeline (source: LinkedIn — Oghli, Build AI RAG Chatbot with Ollama and LangChain)

Building the RAG chatbot

`create_index.py`	Reads dataset/rentals.csv, embeds it with nomic-embed-text, and saves the vector index to saved_index/ — run once.
`query_index.py`	Loads that saved index, retrieves relevant rows for your question, and asks llama3.2 to answer using them — run interactively.

Try it:

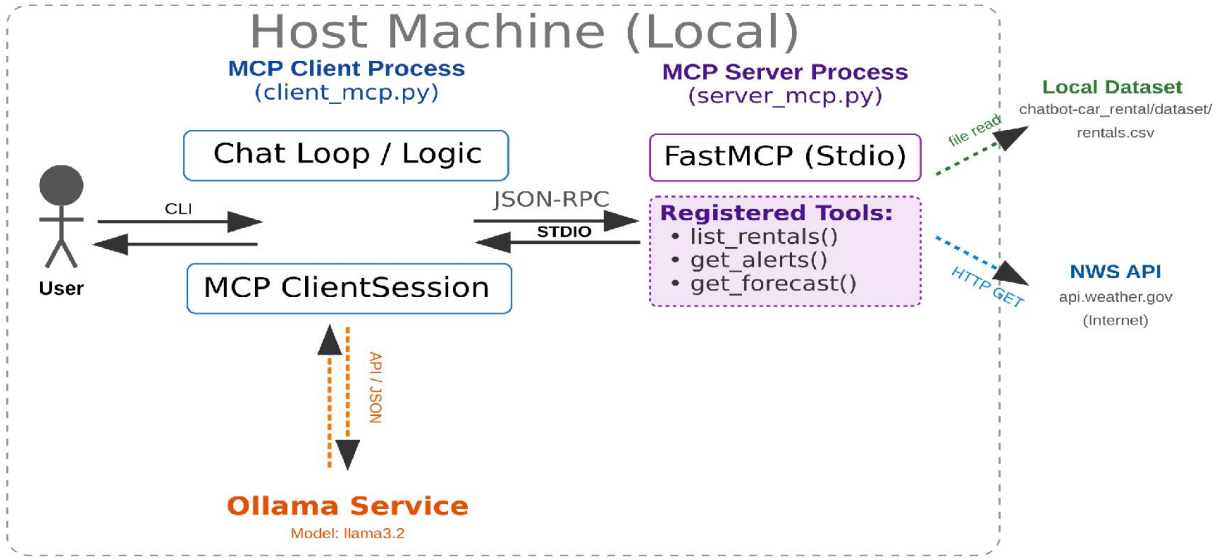
"Can you check which engine types have a low rating?"

⚠️ **Keep saved_index/ — section 3's RAG agent reuses this exact file.**

Model Context Protocol (MCP)

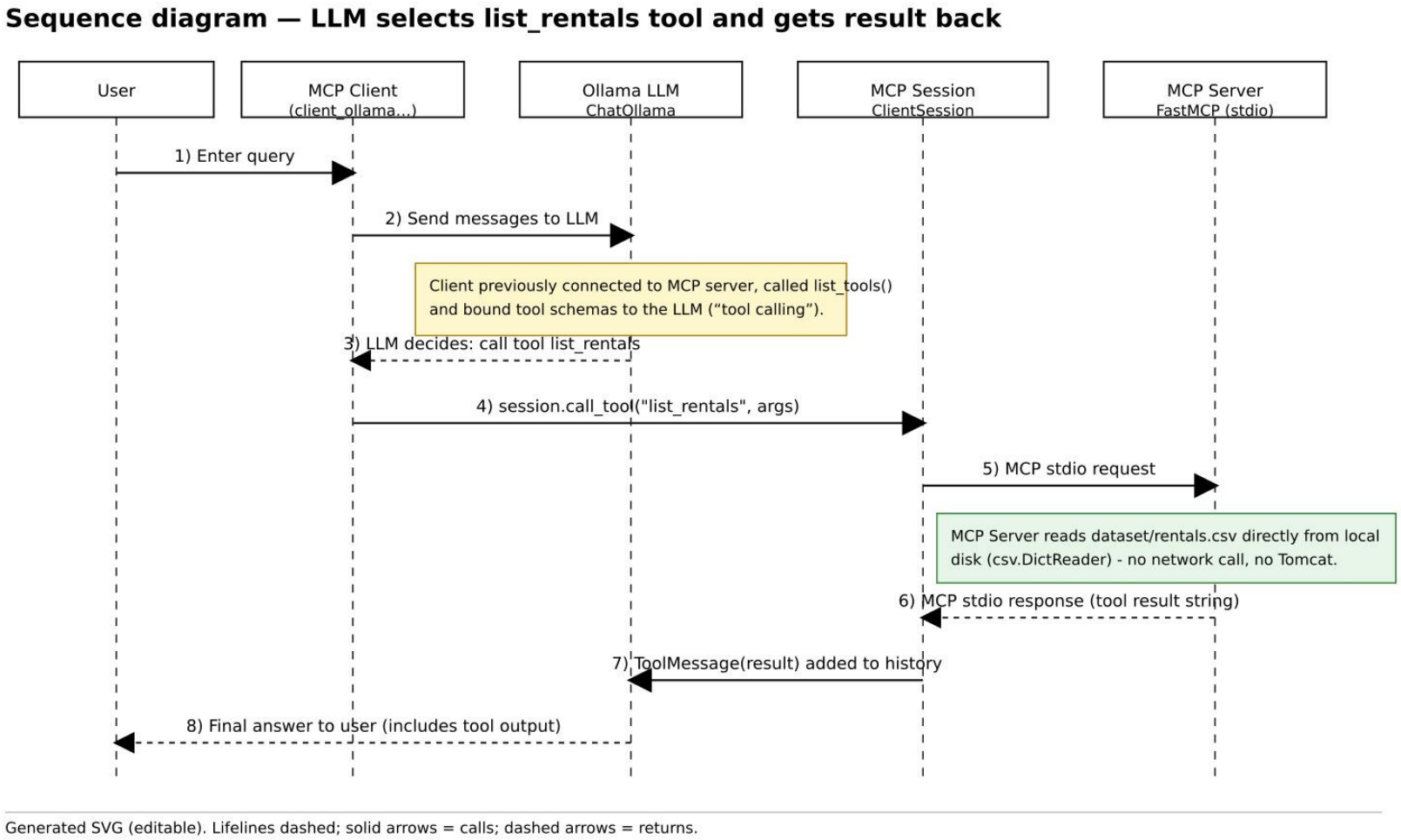
RAG gives the LLM a **frozen snapshot** of data. **MCP** gives it **live tools** it can call on demand instead:

- An MCP server registers callable tools — here, list_rentals (read the CSV live) plus two weather tools.
- An MCP client connects to both the LLM and the server, tells the LLM which tools exist, and relays the LLM's chosen calls to the server.
  - The LLM picks which tool to call and with what arguments — it never talks to the tool directly.
  - **RAG vs MCP: RAG = indexed snapshot; MCP = fresh data fetched at ask-time**.



Chatbot ↔ MCP client ↔ MCP server ↔ tools

Running the MCP chatbot



MCP client/server request sequence

Run it:

```
python3 client_mcp.py server_mcp.py
```

- Confirm all 3 tools are listed: list_rentals, get_alerts, get_forecast.
- Ask: "which rental had the highest number of days?"
- Watch list_rentals get called — that's the LLM choosing a tool, not you.
- Compare this flow to the diagram on the left.

Decentralized Identifiers (DID)

Section 3 **turns each chatbot into a standalone agent** process with its own cryptographic identity. That identity is a DID (W3C standard).

DID

A URI, e.g.
did:web:localhost%3A8101:z6Mk...
Just an identifier.

DID Document

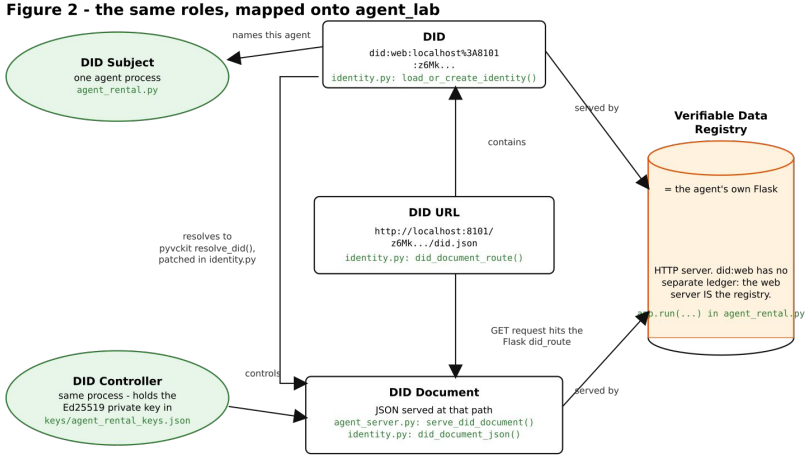
What the DID resolves to: a JSON file l**isting the public key(s)** that prove control of it.

did:web method

This lab's DID method: resolving the DID is literally an HTTPS GET — no blockchain.

Key idea in this lab: every agent creates its own keypair and DID locally, the first time it runs — no registration authority involved.

Figure 2 - the same roles, mapped onto agent_lab



Every agent (trust_anchor.py, agent_rental.py, agent_rag.py, agent_orchestrator.py) plays this same role for its own identity; config.py assigns each its host:port, and common/agent_server.py wires the Flask routes shown above.

Verifiable Credentials (VC)

A DID proves who you are. A Verifiable Credential **proves what you're certified to do** — signed by someone the verifier already trusts.



Credential fields that matter:

- **issuer.id**
  - → the trust anchor's DID (who signed this)
- **credentialSubject.id**
  - → the DID this credential is about
- **credentialSubject.capabilities**
  - → e.g. ["list_rentals"] — what's authorized
- **proof.jws**
  - → the Ed25519 signature over the whole document

⚠️ Pitfall worth remembering: a JSON-LD signature only protects fields your @context maps. Custom fields like capabilities need an @vocab fallback — otherwise they're silently dropped before signing, and tampering with them would go undetected. This lab's LAB_CONTEXT adds that fallback for you.

Four processes, four identities

Each runs as its own process on its own port, with its own DID:

port 8100	port 8101	port 8102	port 8103
`trust_anchor.py`	`agent_rental.py`	`agent_rag.py`	`agent_orchestrator.py`
While running, only serves its own did.json. The public key that lets verifiers check any credential it signs. Signing itself happens separately: issue_credentials.py (a one-shot script) uses its private key to sign each agent's credential.	Offers list_rentals — the successor to section 2's MCP tool.	Offers rag_query, reusing section 1's saved vector index.	The "front door". Discovers peers, verifies their credentials, and routes your question to the right one.

Startup order: trust_anchor → agent_rental / agent_rag → issue_credentials.py → agent_orchestrator

Trust-checked routing — 4 checks before every call

Before the orchestrator calls an agent's /invoke, it runs verify_capability_credential(). All four checks must pass, in order:

1. Signature valid?	2. Trusted issuer?	3. Subject matches?	4. Capability listed?
The credential's proof.jws really was signed by the key in proof.verificationMethod, over an unmodified document.	issuer.id must equal the trust anchor's DID the orchestrator itself is configured to trust. Not just any valid signer.	credentialSubject.id must equal the DID the agent claims in its own agent-card.json — catches a copied credential served by an impostor.	credentialSubject.capabilities must actually contain the capability being invoked (e.g. list_rentals).

Try both routes:

- "list the current car rentals" → routes to agent_rental
- "which engine types have a low rating?" → routes to agent_rag

REQUIRED · SECTION 3.4

Now break it on purpose

1. Stop agent_rental.py.
2. Hand-edit credentials/agent_rental.json — e.g. change "list_rentals" to "list_everything", or flip one character inside proof.jws.
3. Restart agent_rental.py.
4. Ask the orchestrator a rentals question again.

Expected result: the orchestrator refuses the call and prints why — this is the actual security property the trust framework buys you.

This hands-on result — what you actually observe when you tamper with the credential — is what your lab report needs. You do not need to reproduce the "What the four checks actually mean" deep-dive from the README (optional background reading) — just report what happened.

Optional extensions & reporting

Section 4 — Proof-of-possession

- A credential/DID document is just a copyable file. Nothing yet proves the server you're talking to controls the matching private key.
- Complete verify_proof() to check a signed nonce challenge (POST /prove).
- Break it again: check against the wrong nonce → should FAIL.

Section 5 — Self-proposed extension

- Add a 4th agent with a new capability.
- Credential revocation, or validFrom/validUntil expiry checks.
- Mutual authentication: agents also verify the orchestrator's identity.

Reporting / submission

- A tarball of agents_lab/. Excluding env* venvs and generated keys/ / credentials/.
- A PDF report covering the required core (sections 0-3), with screenshots, including the discovery/trust-check log lines and the section 3.4 "break it" result.
- If attempted: sections 4/5 results as extra report sections, including section 4's "break it" result.

Points: RAG (3) + MCP (3) + trusted agent network (4) (core, required). Do the required core first, in order.